Përmbajtja

1 Hyrje në algoritme dhe programim **1**
1.1 Hyrje në algoritme . . . 1
1.2 Vetitë kryesore të një algoritmi . . . 2
1.3 Shembuj nga jeta reale . . . 3
1.4 Pseudokodi . . . 4
1.5 Elementet bazë të pseudokodit . . . 5
1.6 Flowchart-et . . . 7
1.7 Strukturat bazë të algoritmeve . . . 7
1.8 Programi Flowgorithm . . . 8
1.9 Simbolet kryesore të flowchart-it . . . 8
1.10 Ushtrime . . . 11
1.11 Detyrat e shtëpisë . . . 14
1.12 Përmbledhje . . . 16

2 Hyrje në gjuhën C **17**
2.1 Pse po mësojmë C-në? . . . 17
2.2 Prejardhja historike e gjuhës C . . . 17
2.3 Standardizimi i gjuhës C . . . 18
2.4 Familja e gjuhëve të bazuara në C . . . 19
2.5 Pse ia vlen të mësoni C-në sot? . . . 19
2.6 Nga kodi burimor te programi ekzekutues . . . 20
2.7 Forma e përgjithshme e një programi në C . . . 22
2.8 Funksioni `printf` . . . 22
2.9 Sekuencat escape . . . 24
2.10 Leximi i të dhënave: funksioni `scanf` . . . 25
2.11 Operatorët aritmetikë . . . 26
2.12 Ushtrime . . . 28
2.13 Detyrat e shtëpisë . . . 33
2.14 Përmbledhje . . . 34

Appendices **35**

A Detyrat e shtëpisë - Zgjidhjet **37**
A.1 Hyrje në algoritme dhe programim . . . 37
A.2 Hyrje në gjuhën C . . . 41

B Laboratorët **45**

Figurat

Tabelat

2.1 Format specifikuese 23
2.2 Sekuencat `escape` të zakonshme në gjuhën C 24
2.3 Specifikuesit më të përdorur me `scanf` 25
2.4 Operatorët aritmetikë në gjuhën C 26

KAPITULLI 1

Hyrje në algoritme dhe programim

1.1 Hyrje në algoritme

Që të mësoni të programoni mirë, më e rëndësishme se vetë sintaksa e një gjuhe (si Python) është mënyra se si *mendoni* për problemet. Programimi i mirë fillon gjithmonë me një algoritëm të qartë. Përpara se të japim një përkufizim formal, ndaloni dhe mendoni:

- Si do ta shpjegoje me fjalët e tua: *"Çfarë është një algoritëm?"*
- Kur dëgjon shprehjen *"hap pas hapi"*, çfarë të vjen në mendje?
- Jep një shembull nga jeta e përditshme ku ndjek udhëzime të renditura.
- A mendon se algoritmet ekzistojnë vetëm në kompjuter?

Nuk ka rëndësi nëse përgjigjja jote fillestare është e pasaktë apo joformale. Qëllimi i këtyre pyetjeve është të kuptosh që:

- një algoritëm është gjithmonë i lidhur me **hapa të renditur**;
- këta hapa duhet të jenë **të qartë**, jo të dykuptimtë;
- të çojnë në një **qëllim të saktë** (zgjidhje problemi).

Përkufizim — Algoritmi

Algoritëm quajmë një **bashkësi të fundme** hapash **të qartë**, **të saktë** dhe **të renditur**, që përshkruajnë mënyrën se si të zgjidhet një problem ose të kryhet një detyrë. Duke nisur nga të dhënat hyrëse (*input*), ekzekutimi i këtyre hapave prodhon gjithmonë (për input-e të vlefshme) rezultate dalëse (*output*) të parashikueshme dhe korrekte.

Pra, mund ta shihni algoritmin si:

- **recetën logjike** që tregon çfarë duhet bërë;
- **plani i zgjidhjes** që vjen *përpara* programit.

Vetë **programi** (p.sh. në Python) është thjesht zbatimi teknik i algoritmit në një gjuhë që e kupton kompjuteri. Nëse algoritmi është i pasaktë ose i paqartë, atëherë edhe programi që del prej tij do të jetë i pasaktë ose i paqartë, pavarësisht sa "i bukur" duket kodi.

Pse algoritmet janë thelbësore në programim

Shumë fillestarë përpiqen të shkruajnë direkt kod pa menduar algoritmin. Kjo çon në:

- gabime logjike (programi ekzekutohet, por jep përgjigje të gabuara);
- kod të ngatërruar që nuk kuptohet as nga vetë autori pas disa ditësh;
- vështirësi në rregullim dhe zgjerim të programit.

1.2 Vetitë kryesore të një algoritmi

Jo çdo listë udhëzimesh është algoritëm i mirë. Për të qenë i dobishëm në programim, një algoritëm duhet të plotësojë disa veti themelore. Këto veti na ndihmojnë të dallojmë mes:

"një përshkrimi të përgjithshëm" dhe *"një procedure të saktë algoritmike"*.

1. Fundi

Algoritmi duhet të përfundojë pas një numri të kufizuar hapash. Nëse hapat përsëriten pafund, nuk kemi algoritëm të pranueshëm. **Shembull** jo i vlefshëm:

"Merri numrat 1, 2, 3, ... dhe vazhdo t'i shtosh përgjithmonë."

Kjo procedurë nuk ka fund, ndaj nuk mund të quhet algoritëm i mirë.

2. Renditja e qartë

Hapat duhet të kenë rend logjik: dihet qartë cili është hapi i parë, i dytë, i tretë, etj. Rendi i gabuar mund të prishë plotësisht rezultatin. **Shembull** i keq:

"Piq byrekun, më pas ndiz furrën."

Renditja nuk ka kuptim; pa rend të qartë, nuk mund të flasim për algoritëm korrekt.

3. Qartësia

Çdo hap duhet të jetë i formuluar në mënyrë që interpretohet njësoj nga të gjithë (ose nga kompjuteri). Fjalë si "pak", "sa të duket", "ndonjëherë" janë të papranueshme në një algoritëm.

- Jo algoritmik: "Hidh pak kripë."
- Algoritmik: "Hidh 5 gram kripë." ose "Hidh 1 lugë çaji kripë."

Kompjuteri nuk "supozon" asgjë; ai ndjek verbërisht hapat që i jep.

4. Ekzekutueshmëria

Çdo hap duhet të jetë i realizueshëm me mjetet dhe njohuritë e disponueshme.

- Në jetën reale: nëse i jep një fëmije algoritëm që përfshin hapin "zgjidh këtë ekuacion diferencial", ai nuk është ekzekutues për të.
- Në programim: një hap duhet të mund të përkthehet në instruksione të gjuhës së programimit dhe në operacione që kompjuteri mund t'i kryejë realisht.

5. Korrektësia

Për çdo input të vlefshëm, algoritmi duhet të prodhojë rezultatin e duhur. Nëse një procedurë jep përgjigje të gabuara edhe pse ndiqet saktë, atëherë algoritmi është i gabuar. **Shembull:**

"Për të gjetur mesataren e tre numrave, shtoji dhe shumëzoi me 3."

Ky përshkrim *nuk* është algoritëm korrekt, sepse formula është e gabuar.

6. Input-et dhe output-et e qarta

Duhet të jetë e specifikuar:

- çfarë të dhënash merr algoritmi (*input*);
- çfarë rezultatesh jep (*output*).

Shembull (mesatarja e tre notave):

- **Input:** tri nota numerike n_1, n_2, n_3.
- **Output:** një numër $M = \dfrac{n_1 + n_2 + n_3}{3}$.

Algoritmi vs. Programi

Një algoritëm mund të shkruhet:

- me fjali në gjuhë natyrore (si shembujt e mësipërm),
- me pseudokod,
- me diagram *flowchart*,
- me tabela vendimmarrjeje, etj.

Programi është kur këtë algoritëm e përkthejmë në një gjuhë të caktuar programimi (Python, C, Java, etj.). Nëse algoritmi është i paqartë ose i gabuar, programi do të trashëgojë të njëjtat probleme. Prandaj, projektimi i algoritmit është hapi më kritik.

1.3 Shembuj nga jeta reale

Përpara se të shkruajmë algoritme për kompjuterin, është e dobishme të shohim se si algoritmet shfaqen në situata të zakonshme. Kjo ju ndihmon të ndërtoni intuiten se *"të menduarit algoritmik"* nuk është diçka artificiale, por diçka që tashmë e përdorni.

Shembulli 1: Receta e gatimit

Input: përbërësit (miell, vezë, qumësht, vaj, kripë, etj.).

```
Mat sasinë e nevojshme të përbërësve.
Hidh përbërësit në enë.
Përziej derisa masa të bëhet homogjene.
Ndez furrën në temperaturën e caktuar.
Hidh masën në tepsi.
Fute tepsinë në furrë dhe piqe për t minuta.
Nxirre nga furra dhe shërbe.
```

Output: ushqimi i përgatitur.

Ky është algoritëm i mirë sepse: **(1)** ka fillim dhe fund të qartë (fundorësi); **(2)** hapat janë të renditur logjikisht; **(3)** hapat janë të ekzekutueshëm; **(4)** për të njëjtin input dhe ndjekje korrekte të hapave, rezultati është i parashikueshëm.

Shembulli 2: Rregullat e një loje me zar

Input: pozicioni fillestar i lojtarëve, një zar.

```
Për çdo lojtar me radhë:
   Hidh zarin.
   Nëse del 6:
      lëviz 2 herë.
   Përndryshe:
      lëviz aq hapa sa numri në zar.
   Nëse lojtari ka arritur fundin e tabelës:
      loja përfundon, ky lojtar fiton.
Përndryshe:
   kalo tek lojtari tjetër dhe përsërit.
```

Output: emri i lojtarit fitues.

Ky shembull tregon **(1)** përdorimin e **kushtit** (`if del 6`); **(2)** përdorimin e **përsëritjes** (loja vazhdon në rreth); **(3)** një **kriter përfundimi** të qartë (dikush fiton).

Shembulli 3: Kalimi i rrugës

Ky shembull është klasik për të shpjeguar vendimmarrjen dhe ciklin.

```
Afrohu te trotuari.
Shiko majtas dhe djathtas.
Nëse nuk po vjen asnjë makinë:
   kalo rrugën deri në anën tjetër.
   algoritmi përfundon.
Përndryshe:
   prit disa sekonda.
   kthehu te hapi "Shiko majtas dhe djathtas".
```

Pikat kyçe:

- Hapi "Nëse nuk po vjen asnjë makinë" është një **kusht** (decision).
- Kthimi te hapi i mëparshëm formon një **loop** (përsëritje).
- Procesi është **fundor**: sapo rruga të jetë bosh dhe të kalosh, algoritmi mbaron.

Çfarë duhet të mbash mend si student?

Nga ky seksion, është shumë e rëndësishme të keni të qartë se:

1. Algoritmi është **zemra** e çdo programi.
2. Algoritmi duhet të jetë i **qartë**, i **saktë**, i **fundëm** dhe i **ekzekutueshëm**.
3. Shembujt e përditshëm (recetat, lojërat, rregullat, procedurat) mund të shkruhen si algoritme.
4. Në kapitujt vijues, këto ide do t'i kthejmë në **pseudokod**, **flowchart-e** dhe më pas në **programe konkrete** në Python.

Nëse mësoni të mendoni në mënyrë "hap pas hapi" që tani, do të keni një avantazh shumë të madh në pjesën tjetër të kursit.

1.4 Pseudokodi

Deri tani kemi folur për algoritmet me fjalë të zakonshme dhe shembuj nga jeta reale. Hapi i radhës, përpara se të kalojmë në gjuhë programimi (si Python), është të përdorim **pseudokodin**. **Pseudokodi** është një mënyrë e strukturuar për të shkruar algoritme që duket si kod, por *nuk* i përket asnjë gjuhe specifike programimi. Ai qëndron "në mes": midis gjuhës natyrore (shumë e lirshme, shpesh e paqartë) dhe gjuhës së programimit (shumë e saktë, me rregulla strikte sintakse).

Përkufizim — Pseudokodi

Pseudokod quajmë një mënyrë të shkruarit të algoritmeve me instruksione të thjeshta, të lexueshme nga njeriu, që ngjasojnë me komandat e një gjuhe programimi, por **pa u kufizuar** nga sintaksa formale e asaj gjuhe. Qëllimi nuk është që pseudokodi të ekzekutohet nga kompjuteri, por që algoritmi të kuptohet qartë nga njerëzit dhe të përkthehet lehtë në kod.

Pseudokodi është një mjet shumë i dobishëm në procesin e të menduarit algoritmik:

- **Qartësi.** Na lejon të fokusohemi te *logjika* e zgjidhjes, pa u hutuar nga presjet, kllapat dhe rregullat teknike të gjuhës.
- **Afër kodit.** Është mjaft i ngjashëm me kodin sa që mund të kthehet lehtë në një program real.
- **Pa sintaksë rigoroze.** Nuk ka "gabime sintakse"; rëndësi ka që hapat të jenë të qartë dhe të saktë.
- **Komunikim.** Është i lehtë për t'u kuptuar nga kushdo që di bazat e algoritmeve, pavarësisht gjuhës së programimit që përdor.

Pseudokodi është veçanërisht i dobishëm për studentët: së pari siguron që zgjidhja logjike është korrekte, pastaj lehtësohet kalimi në Python ose gjuhë të tjera. Edhe pse nuk ka rregulla strikte, një pseudokod i mirë duhet:

- të ketë **hapa të mirëpërcaktuar** (pa dykuptimësi, pa terma si "pak", "ndoshta");
- të ndjekë një **strukturë të qartë** (rendi i hapave, blloqe `NËSE`, `PËRSËRIT`, etj.);
- të jetë **i thjeshtë për t'u lexuar**;
- të jetë **i lehtë për t'u kthyer në kod**.

1.5 Elementet bazë të pseudokodit

Në këtë libër do të përdorim një stil pseudokodi të thjeshtë, në shqip, me fjalë kyçe intuitive. Qëllimi është që ju ta lexoni sa më natyrshëm, por njëkohësisht të mësoheni me konceptet themelore që do t'i gjeni edhe në gjuhët e programimit. Elementet kryesore të pseudokodit janë:

1. Operacionet sekvenciale (hap pas hapi)

Instruksionet ekzekutohen në rendin që janë shkruar, njëri pas tjetrit.

```
Lexo A
Lexo B
Shuma = A + B
Shfaq Shuma
```

Kjo quhet **sekuencë**. Është struktura më e thjeshtë dhe baza e çdo algoritmi.

2. Leximi i input-it

Instruksionet për të marrë të dhëna nga përdoruesi ose nga një burim tjetër.

```
Lexo A
Lexo B
```

3. Shfaqja e output-it

Instruksionet për të treguar rezultatin.

```
Shfaq Shuma
Shfaq "Rezultati është:", Shuma
```

4. Veprimet aritmetike dhe caktimi i vlerave

Llogaritjet themelore dhe ruajtja e rezultateve në variabla:

```
Shuma = A + B
Mesatarja = (A + B + C) / 3
Diferenca = A - B
```

5. Operacionet kushtore (vendimmarrja)

Për të marrë vendime në bazë të një kushti përdorim strukturën **NËSE**.

```
Nëse A > B atëherë
   Shfaq "A është më i madh se B"
Përndryshe
   Shfaq "A nuk është më i madh se B"
FundNese
```

6. Operacionet e përsëritura (ciklet)

Për veprime që duhet të përsëriten disa herë përdorim *loops* (cikle). Në pseudokod mund të shkruajmë, për shembull:

```
Për i nga 1 deri në 5 bëj
   Lexo numër
   Shfaq numër
FundPër
```

ose:

```
Përsërit
   Lexo numër
derisa numër == 0
```

Këto struktura na paralajmërojnë për `for` dhe `while` që do të mësoni në një nga gjuhët e programimit.

Rëndësia e elementeve bazë

Çdo algoritëm mund të ndërtohet nga kombinimi i këtyre tre strukturave kryesore:

1. **Sekuenca** (hap pas hapi),
2. **Zgjedhja** (kushtet NËSE),
3. **Përsëritja** (ciklet).

Pseudokodi është "laboratori" ku i ushtrojmë këto ide përpara se t'i kthejmë në kod real.

Algoritmi: Shuma e dy numrave

Dua të shkruaj një procedurë që: **(1)** merr dy numra nga përdoruesi; **(2)** llogarit shumën e tyre; **(3)** shfaq rezultatin.

```
Start
Lexo numrin e parë dhe ruaje në A
Lexo numrin e dytë dhe ruaje në B
Shuma = A + B
Shfaq "Shuma e dy numrave është:", Shuma
End
```

- **Start / End** tregojnë fillimin dhe fundin e algoritmit.
- **Lexo A, Lexo B** janë hapat e *input*-it. Pa këto, algoritmi nuk di me çfarë vlerash të punojë.
- **Shuma = A + B** është një veprim aritmetik dhe caktim vlerash: ne krijojmë një variabël (Shuma) dhe i japim vlerën e përllogaritur.
- **Shfaq ...** është *output*-i: pa këtë hap, përdoruesi nuk sheh rezultatin.

Nga pseudokodi në program

Ky pseudokod mund të përkthehet pothuajse *rresht më rresht* në një gjuhë programimi. Kjo është arsyeja pse pseudokodi është kaq i fuqishëm: ai ju mëson të mendoni si programues, pa u ngatërruar ende me detaje teknike. Në kapitujt vijues do të marrim pseudokode si ky dhe do t'i kthejmë në programe funksionale.

1.6 Flowchart-et

Deri tani kemi parë dy mënyra për të përshkruar një algoritëm:

- me fjali në gjuhë natyrore;
- me pseudokod.

Një mënyrë tjetër shumë e përdorur, veçanërisht për të ilustruar algoritme për fillestarët, është paraqitja grafike me anë të **flowchart**-eve.

Përkufizim — Flowchart

Flowchart quajmë një **diagram grafik** që paraqet hapat e një algoritmi përmes **simbolesh** (forma gjeometrike) dhe **shigjetash** që tregojnë drejtimin e ekzekutimit. Çdo simbol përfaqëson një lloj veprimi (fillim/fund, proces, vendim, input, output, etj.), ndërsa shigjetat tregojnë rendin "hap pas hapi".

Pra, një flowchart është thjesht *algoritmi i vizatuar*. Ai nuk ndryshon logjikën e algoritmit — thjesht e bën më të lehtë: (1) për t'u kuptuar vizualisht; (2) për t'u diskutuar në klasë; (3) për të parë qartë degëzimet dhe përsëritjet; (4) për të kontrolluar nëse kemi harruar ndonjë rast.

1.7 Strukturat bazë të algoritmeve

Pavarësisht kompleksitetit, shumica e algoritmeve ndërtohen nga të njëjtat tre "blloqe logjike". T'i njohim që tani do t'ju ndihmojë shumë në kapitujt vijues.

1. Sequence (Sekuenca)

Sequence do të thotë: instruksionet ekzekutohen njëri pas tjetrit, në rendin në të cilin janë shkruar.

```
Lexo numrin e parë dhe ruaje në A
Lexo numrin e dytë dhe ruaje në B
Shuma = A + B
Shfaq "Shuma e dy numrave është:", Shuma
```

Edhe kur ka kushte, brenda çdo dege hapat janë sërish sekuencialë.

2. Selection (Zgjedhja / Branching)

Selection përdoret kur algoritmi duhet të zgjedhë mes dy (ose më shumë) rrugëve në varësi të një kushti.

```
Nëse nota >= 50 atëherë
   Shfaq "Kalon"
Përndryshe
   Shfaq "Nuk kalon"
FundNese
```

3. Repetition (Përsëritja / Loops)

Repetition përdoret kur një grup hapash duhet të ekzekutohet disa herë: një numër i caktuar herësh, ose derisa të plotësohet një kusht.

```
Për i nga 1 deri në 5 bëj
   Lexo numër
   Shfaq numër
FundPër
```

```
Përsërit
   Lexo numër
derisa numër == 0
```

1.8 Programi Flowgorithm

Për të vizatuar flowchart-e në mënyrë të rregullt dhe profesionale,në laborator mund të përdorim programe specifike. Një prej tyre është **Flowgorithm**. Roli i këtij programi është thjesht:

- të ofrojë simbolet standarde (Start/End, Process, Input/Output, Decision, etj.);
- të lejojë vendosjen e tyre dhe lidhjen me shigjeta;
- të ndihmojë në krijimin e flowchart-eve të pastra dhe të lexueshme.

Është e rëndësishme të kuptohet se:

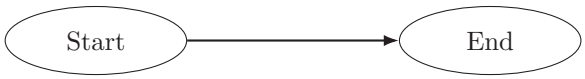
- Flowgorithm **nuk e zbulon algoritmin për ju**;
- së pari duhet të keni menduar algoritmin (me fjalë ose pseudokod), më pas e vizatoni me mjetin grafik.

1.9 Simbolet kryesore të flowchart-it

Për të lexuar dhe ndërtuar flowchart-e në mënyrë të saktë, duhet të njohim simbolet kryesore standarde. Çdo simbol ka kuptim të caktuar dhe përdoret në mënyrë të qëndrueshme.

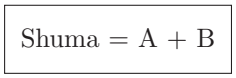
1. Start / End (Fillim / Mbarim)

Përdorimi: Tregon fillimin ose përfundimin e algoritmit.



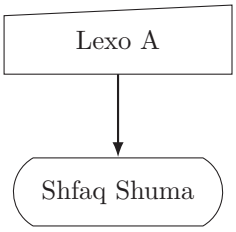
2. Process (Proces)

Përdorimi: Hapat e përpunimit / llogaritje.



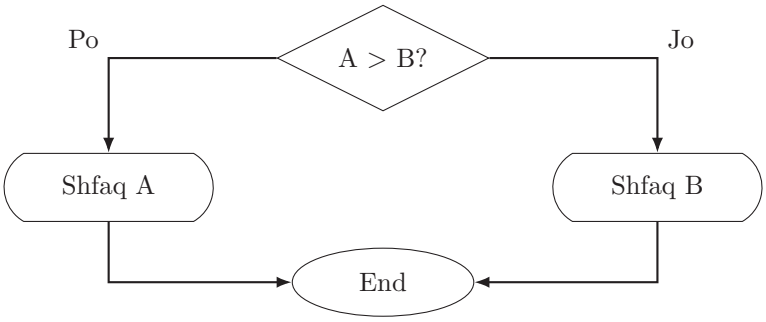
3. Input / Output

Përdorimi: Lexim të dhënash (Input) ose shfaqje rezultatesh (Output).



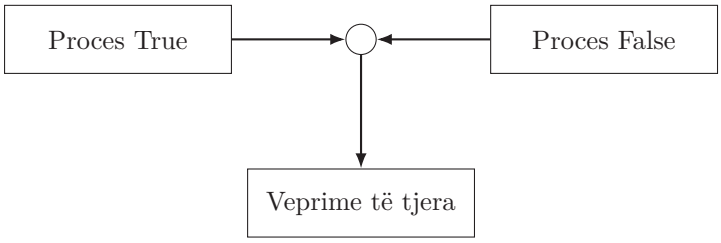
4. Decision (Vendim)

Përdorimi: Pyetje logjike me degëzim **Po/Jo** (**True/False**).



5. Connector (Lidhës)

Përdorimi: Lidh dy pjesë të largëta të të njëjtit flowchart (p.sh. në faqe të ndryshme). Etiketa e njëjtë (p.sh. "A") tregon vazhdimësinë e rrjedhës.



Si t'i lexosh simbolet

Kur sheh një flowchart:

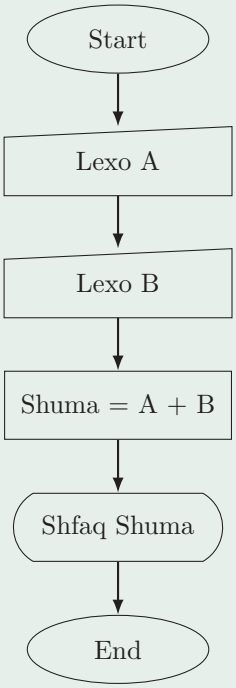
- fillo nga **Start** dhe ndiq shigjetat;
- në çdo **Process**, mendo: "Çfarë po llogaritet këtu?";
- në çdo **Input/Output**, mendo: "Çfarë hyn? Çfarë del?";
- në çdo **Decision**, mendo: "Cilat janë degët Po/Jo? Çfarë ndodh në secilën?"

Kjo korrespondon direkt me strukturat: **sekuencë**, **zgjedhje**, **përsëritje**.

Shembull: Shuma e dy numrave si flowchart

Tani ta lidhim gjithçka në një shembull të plotë: nga ideja, te pseudokodi, te flowchart-i. I njëjti algoritëm që pamë në pseudokod mund të paraqitet si flowchart.

```
Start
Lexo numrin e parë dhe ruaje në A
Lexo numrin e dytë dhe ruaje në B
Shuma = A + B
Shfaq "Shuma e dy numrave është:", Shuma
End
```



Flowchart-i tregon qartë rrugën nga input-et (A, B), te llogaritja e shumës dhe shfaqja e rezultatit. Këtu shihen qartë:

- simbolet Start/End për kufijtë e algoritmit;
- dy simbole Input për leximin e A dhe B;
- një simbol Process për llogaritjen;
- një simbol Output për shfaqjen e rezultatit.

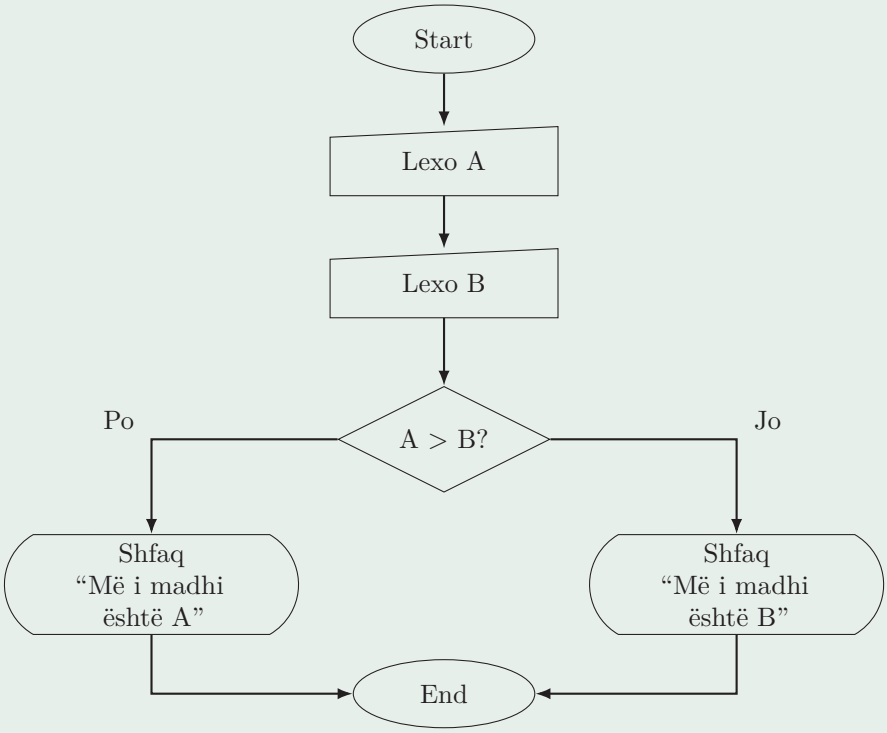
Shembull: Gjetja e numrit më të madh

Tani ndërtojmë një algoritëm ku kemi një **vendim** (Decision) të qartë. Kjo na ndihmon të kuptojmë më mirë degëzimin **True/False**. Algoritmi duhet:

- të lexojë dy numra A dhe B;
- të krahasojë cilin prej tyre është më i madh;
- të shfaqë numrin më të madh.

```
Start
Lexo A
Lexo B
Nëse A > B atëherë
   Shfaq "Numri më i madh është:", A
Përndryshe
   Shfaq "Numri më i madh është:", B
FundNese
End
```

Vëre këtu: **(1)** përdorim strukturën **Selection** (zgjedhje); **(2)** kushti `A > B` përcakton cilën degë ekzekutojmë; **(3)** gjithmonë shfaqim një përgjigje të vetme, të qartë.



Vini re:

- Simboli **Decision** mban pyetjen “A > B?”.
- Degët “Po” dhe “Jo” çojnë në dy veprime të ndryshme.
- Të dyja degët bashkohen te **End** — algoritmi mbyllet pas një përgjigjeje.

Fokus te Decision

Simboli **Decision** është thelbësor:

- çdo herë që algoritmi duhet të marrë një vendim *po/jo*, përdor këtë simbol;
- çdo degë duhet të ketë rrjedhë të plotë deri në fund (nuk “varim” shigjeta në ajër);
- në kod, ky simbol do të përkthehet në struktura `if`, `if-else` dhe me rradhë.

Nëse e kupton këtë shembull, je gati të ndërtosh flowchart-e për algoritme më komplekse.

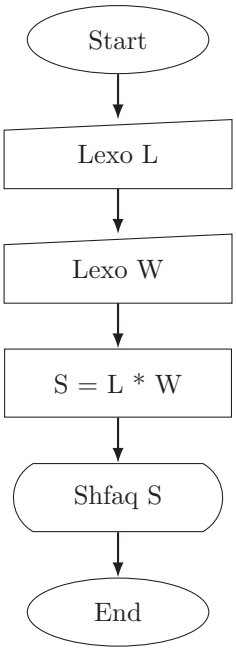
1.10 Ushtrime

Ushtrim 1: Sipërfaqja e drejtkëndëshit

Shkruaj algoritmin për të llogaritur sipërfaqen S të një drejtkëndëshi.

Input: gjatësia L dhe gjerësia W.
Output: vlera $S = L \times W$.

```
Start
Lexo L
Lexo W
S = L * W
Shfaq "Sipërfaqja është:", S
End
```

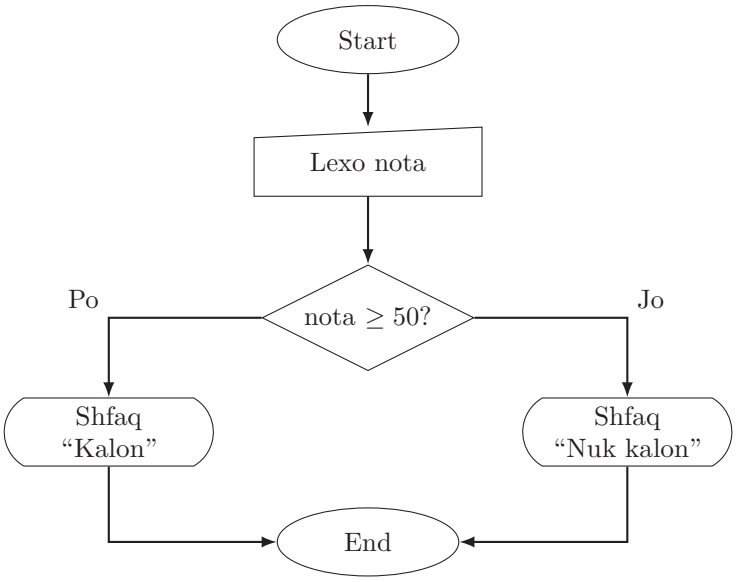


Ushtrim 2: A kalon studenti?

Të ndërtohet algoritmi që vendos nëse studenti kalon lëndën. Të supozojmë se studenti kalon nëse nota ≥ 50, përndryshe nuk kalon.

Input: nota (`nota`).
Output: mesazhi "Kalon" ose "Nuk kalon".

```
Start
Lexo nota
Nëse nota >= 50 atëherë
   Shfaq "Kalon"
Përndryshe
   Shfaq "Nuk kalon"
FundNese
End
```



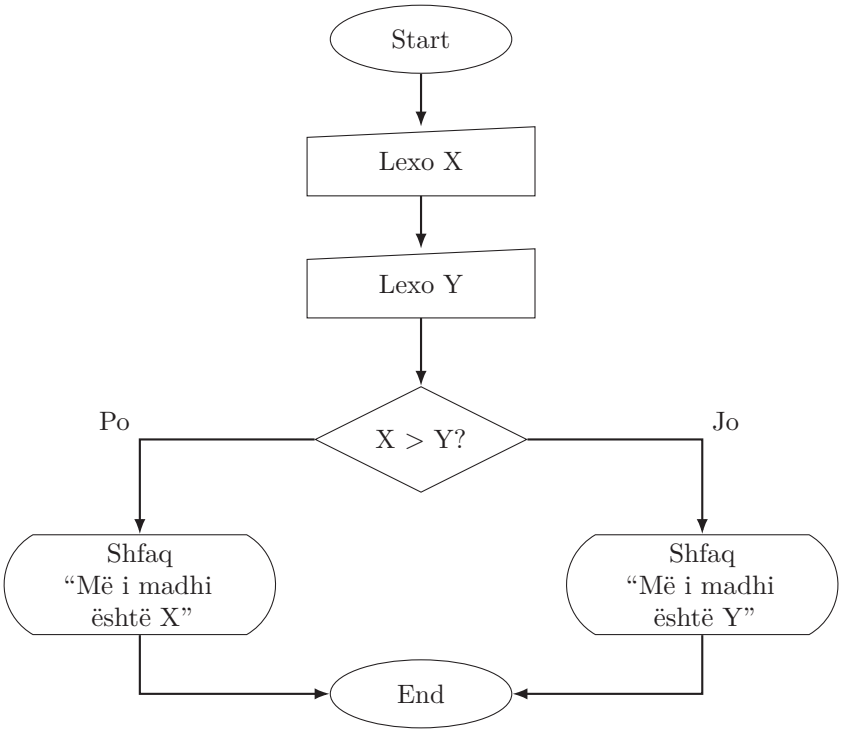
Ushtrim 3: Numri më i madh nga dy vlera

Ndërto flowchart për të gjetur numrin më të madh nga dy vlera X dhe Y. Ky ushtrim është shumë i ngjashëm me shembullin "Më i madhi është A/B" që pamë më sipër. Mund ta përdorni:

- si ripërsëritje të strukturës **Decision**;
- si verifikim nëse kuptoni si bashkohen degët **Po/Jo** te fundi i algoritmit.

Input: numri (X) dhe (Y).
Output: mesazhi "Më i madhi është X" ose "Më i madhi është Y".

```
Start
Lexo X
Lexo Y
Nëse X > Y atëherë
   Shfaq "Numri më i madh është:", X
Përndryshe
   Shfaq "Numri më i madh është:", Y
FundNese
End
```



Diskutim i shpejtë i zgjidhjeve

Kur kontrolloni zgjidhjet (në klasë ose vetë), verifikoni:

- A ka **gjithmonë** një **Start** dhe një **End**?
- A janë të gjitha shigjetat të lidhura saktë (pa linja që "varen" në ajër)?
- A përdoret simboli i duhur për çdo lloj hapi (Input, Output, Process, Decision)?
- A mbulohen të gjitha rastet? (p.sh. si sillet algoritmi kur nota është saktësisht 50?)

Nëse këto kushte mbahen, flowchart-i është në rrugë të mbarë.

1.11 Detyrat e shtëpisë

Detyra 1: A është algoritëm i mirë?

Për secilin prej përshkrimeve më poshtë:

a) vendos nëse është **algoritëm i vlefshëm** apo **jo**;

b) në 1–2 rreshta shpjego pse (duke përdorur vetitë: fundorësi, qartësi, input/output, etj.).

```
Hap librin ku të duash dhe lexo derisa të lodhesh.
Merr tre numra, gjej shumën e tyre dhe pjesëto me 3. Shfaq rezultatin.
Përsërit: shto 1 numrit fillestar. (pa asnjë kusht ndalimi)
Hidh zarin derisa të dalë 6; numëro sa herë u hodh zar.
```

Detyra 2: Rregullo algoritmin e paqartë

Jepet ky përshkrim për përgatitjen e një pijeje:

> "Mbush gotën me pak ujë, shto pak lëng, përziej mirë dhe shërbeje të ftohtë."

a) Shpjego pse ky përshkrim **nuk** është algoritëm i mirë.

b) Shkruaj një version të përmirësuar si algoritëm (me fjali të qarta).

c) Shkruaj pseudokodin për versionin tënd të përmirësuar.

Detyra 3: Pseudokod dhe flowchart për mesataren

Shkruaj një algoritëm që:

- lexon tre numra A, B, C;
- llogarit mesataren e tyre aritmetike;
- shfaq mesataren.

Kërkohet:

a) shkruaj algoritmin me fjali të thjeshta;

b) jep pseudokodin;

c) vizato flowchart-in duke përdorur simbolet Start/End, Input, Process, Output.

Detyra 4: Vendos nëse studenti kalon (version i zgjeruar)

Studimi kalon nëse:

- nota ≥ 80: `"Shkëlqyeshëm"`
- $50 \leq$ nota < 80: `"Kalon"`
- nota < 50: `"Nuk kalon"`

Kërkohet:

a) shkruaj pseudokodin;

b) vizato flowchart-in me një simbol **Decision** ose disa, duke mbuluar të tre rastet.

Detyra 5: Numri më i madh nga tre numra

Ndërto një algoritëm që lexon tre numra X, Y, Z dhe shfaq numrin më të madh.
Kërkohet:

a) shkruaj pseudokodin duke përdorur struktura **Selection**;

b) vizato flowchart-in përkatës.

Detyra 6: Temperatura e ujit

Shkruaj algoritmin që, për një temperaturë T në gradë Celsius, shfaq gjendjen e ujit:

- nëse $T < 0$: `"Akull"`;
- nëse $0 \leq T \leq 100$: `"Lëng"`;
- nëse $T > 100$: `"Avull"`.

Kërkohet:

a) pseudokodi;

b) flowchart-i me degëzime të qarta.

1.12 Përmbledhje

Në këtë kapitull keni parë tre nivele përshkrimi të së njëjtës ide:

1. **Algoritmi (me fjalë).**
   - Përshkrimi konceptual i zgjidhjes.
   - Qëllimi: të kuptohet *logjika* dhe hapat kryesorë.
2. **Pseudokodi.**
   - Hapa më të strukturuar, të ngjashëm me kodin.
   - Qëllimi: të jetë urë midis mendimit logjik dhe gjuhës së programimit.
   - Lehtëson kalimin drejt implementimit në Python (ose gjuhë të tjera).
3. **Flowchart-i.**
   - Paraqitje grafike e algoritmit me simbole standarde dhe shigjeta.
   - Qëllimi: ta bëjë rrjedhën e ekzekutimit të dukshme dhe të lehtë për t'u ndjekur.

Kur të punoni me detyrat në vijim të kursit:

1. **Mos nxitoni direkt te kodi.** Së pari shkruani algoritmin me fjalë.
2. **Përdorni pseudokod.** Formuloni qartë hapat, kushtet dhe ciklet.
3. **Nëse ndihmon, vizatoni flowchart.** Sidomos kur ka shumë degëzime dhe përsëritje.
4. **Vetëm në fund** përkthejeni këtë logjikë në një program konkret.

Nëse ndiqni këtë rregull, do të shkruani kode më të pastra, më të sakta dhe shumë më të lehta për t'u kuptuar.

KAPITULLI 2

Hyrje në gjuhën C

2.1 Pse po mësojmë C-në?

Nëse kapitulli i parë ju mësoi të mendoni algoritmikisht, në këtë kapitull fillojmë t'i lidhim këto ide me një nga gjuhët më me ndikim në historinë e programimit: **gjuhën C**.

Përkufizim — Gjuha C

C është një gjuhë programimi e nivelit relativisht të ulët (krahasuar me gjuhë si Python apo Java), e tipit *procedurial*, e projektuar fillimisht për zhvillimin e sistemeve operative dhe software-ve të lidhura ngushtë me harduerin. Ajo kombinon:

- qasje efikase në memorje,
- operacione të fuqishme mbi bitët dhe adresat,
- sintaksë kompakte dhe të standardizuar,
- portabilitet në platforma të ndryshme.

Edhe pse shumë prej jush mund të kenë filluar me Python-in (më i thjeshtë për fillestarë), gjuha C ju jep një kuptim më të thellë të:

- si funksionon kompjuteri "poshtë kapakut",
- si menaxhohen manualisht memorja dhe burimet,
- si janë ndërtuar vetë gjuhët e tjera moderne.

Mesazh për studentin

Nëse e mësoni C-në me durim:

- do lexoni dhe kuptoni kod "serioz" të sistemeve reale;
- do jeni më të përgatitur për të kaluar te C++, Java, C#, apo gjuhë të tjera të bazuara në C;
- do kuptoni më mirë kufijtë dhe magjinë e gjuhëve me nivel më të lartë (si Python).

2.2 Prejardhja historike e gjuhës C

Nga Algol te BCPL dhe B

Shumë ide të C-së nuk lindën nga hiçi. Ato rrjedhin nga një linjë gjuhësh më të hershme:

- **Algol 60** — një nga gjuhët më të rëndësishme akademike të viteve 1960; prezantoi struktura të qarta kontrolli dhe ndikoi shumë në dizajnin e gjuhëve moderne.
- **BCPL** (Basic Combined Programming Language) — e zhvilluar në mesin e viteve 1960; një gjuhë e thjeshtë dhe e përshtatshme për sistemet.

- **B** — gjuhë e projektuar nga Ken Thompson në Bell Labs, e bazuar në BCPL; u përdor për versionet e hershme të sistemit operativ UNIX.

Gjuhët Assembly, BCPL dhe B treguan një problem: ose ishin shumë të varura nga një procesor i caktuar (Assembly), ose tepër të kufizuara për harduerë të rinj. Kjo krijoi nevojën për një gjuhë që:

- të ishte mjaftueshëm e **afërt me harduerin**, për të shkruar sistem operativ;
- por edhe mjaftueshëm **e përgjithshme dhe portabël**, që kodi të mund të çohej lehtë nga një makinë në tjetrën.

Lindja e C-së në Bell Labs

Në fillim të viteve 1970, në *Bell Labs*, Dennis Ritchie filloi të zhvillojë një version të zgjeruar të B-së për kompjuterin DEC PDP-11, i cili kishte veçori harduerike që B nuk i shfrytëzonte mirë. Gjuha e re u quajt fillimisht *NB* (*New B*), dhe më pas mori emrin që njohim sot: **C**. Rreth vitit 1973, një pjesë shumë e madhe e sistemit operativ **UNIX** u rishkrua në C. Ky ishte një moment historik:

- sistemi operativ nuk ishte më i lidhur ngushtë me një gjuhë Assembly;
- duke shkruar kompilues C për makina të ndryshme, UNIX mund të portohej më lehtë;
- C u bë mjeti kryesor për zhvillimin e sistemeve në shumë platforma.

Pse kjo histori ka rëndësi për ju

- C është projektuar nga njerëzit që shkruan një sistem operativ real (UNIX), jo si ushtrim teorik.
- Sintaksa dhe filozofia e C-së reflektojnë nevojat praktike të inxhinierëve: efikasitet, kontroll, thjeshtësi në bërthamë.
- Shumë sisteme moderne (Linux, pjesë të Windows-it, drejtues pajisjesh, biblioteka sistemike) ruhen ende në C.

2.3 Standardizimi i gjuhës C

Nga K&R tek C89/C90

Me përhapjen e C-së në vitet 1970, u shfaq nevoja për dokumentim zyrtar. Në vitin 1978 u botua libri: *The C Programming Language* — Brian Kernighan & Dennis Ritchie. Ky libër (i njohur si *K&R*) u bë referenca kryesore për C-në dhe për një kohë të gjatë shërbeu si "standard jozyrtar". Por kishte një problem: për disa veçori të gjuhës përshkrimi ishte i paqartë. Si rezultat:

- kompilues të ndryshëm implementonin gjuhën paksa ndryshe;
- programet C humbnin **portabilitetin**, një nga arsyet kryesore pse C ishte krijuar.

Për ta zgjidhur këtë, në vitin 1983 filloi puna për një standard zyrtar. Rezultati:

- **ANSI X3.159-1989** — i njohur si **C89**;
- më vonë u miratua nga ISO si **ISO/IEC 9899:1990**, i njohur si **C90**.

Çfarë solli C89/C90?

- Përkufizim i qartë i sintaksës dhe semantikës së C-së;
- bibliotekë standarde (standard library) e përcaktuar në mënyrë formale;
- bazë e përbashkët për kompiluesit në platforma të ndryshme.

Që nga ky moment, kur themi "program në C të pastër", nënkuptojmë kod që respekton standardin, jo një dialekt lokal.

Versionet pasuese: C99, C11, C17, C23

Me kalimin e viteve, standardi i C-së është përditësuar disa herë. Në këtë kurs nuk kemi nevojë të futemi në detaje teknike të çdo versioni, por është e dobishme të njihni kronologjinë:

- **C99** (ISO/IEC 9899:1999) — shtoi tipin `long long`, komente të stilit `//`, përmirësime në `for`, inicializues të përparuar, etj.
- **C11** (ISO/IEC 9899:2011) — shtoi veçori për *multithreading*, tipa atomikë, funksione të reja, përmirësime portabiliteti.
- **C17/C18** (ISO/IEC 9899:2018) — kryesisht përmirësime dhe korrigjime (pa ndryshime të mëdha konceptuale).
- **C23** — standard i fundit (në momentin e shkrimit të këtij materiali), me rafinime të tjera të gjuhës dhe bibliotekës.

Çfarë duhet të mbani mend në këtë kurs

Për shumicën e temave të këtij libri:

- do të punojmë me një **nën-bërthamë** të C-së, e cila është e vlefshme në C89, C99, C11, C17, C23;
- do të theksojmë aty ku ka ndryshime të rëndësishme mes standardeve;
- do të shkruajmë kod që kompilohet në çdo kompilues modern C.

2.4 Familja e gjuhëve të bazuara në C

Suksesi i C-së nuk është vetëm në përdorimin e saj direkt, por edhe në ndikimin që ka pasur në gjuhë të tjera moderne.

- **C++** — filloi si "C me klasa"; ruan të gjithë C-në dhe shton: programim të orientuar në objekte, template, abstraksione më të larta.
- **Java** — huazon sintaksën bazë nga C/C++, por ofron menaxhim automatik të memories dhe platformë virtuale.
- **C#** — e zhvilluar nga Microsoft; frymëzuar nga C++ dhe Java, e përdorur gjerësisht për aplikacione desktop, web dhe lojëra.
- **JavaScript** — edhe pse teknikisht shumë ndryshe, sintaksa e saj (if, for, operatorët, blloqet) duket "familjare" për këdo që di C.
- **Perl**, **PHP**, **Go**, **Rust** dhe shumë gjuhë të tjera marrin elemente sintaksore ose konceptuale nga C.

Mesazhi kryesor

Nëse mësoni mirë C-në:

- kalimi në C++, Java, C#, JavaScript, Go, etj. bëhet dukshëm më i lehtë;
- operatorët, strukturat e kontrollit dhe shumë koncepte nuk do jenë më të reja për ju;
- do kuptoni *burimin* e shumicës së idiomave të programimit modern.

2.5 Pse ia vlen të mësoni C-në sot?

Pyetja e natyrshme: *"Me gjithë këto gjuhë moderne, pse të merrem me C-në?"*

1. **Kupton si funksionon kompjuteri.**
   C të detyron të mendosh për:

- llojet e të dhënave,
- adresat e memorjes,
- pointer-at,
- mënyrën si ruhen dhe lëvizin të dhënat.

Ky nivel kontrolli nuk shfaqet kaq qartë në gjuhë me nivel të lartë si Python.

2. **Është gjuhë kyçe për sistemet.**
   Shumë sisteme operative, drejtues pajisjesh (*drivers*), kompajlerë, interpretuese, biblioteka standarde janë shkruar (ose kanë pjesë kritike) në C.

3. **Portabilitet dhe jetëgjatësi.**
   Kodi në C mund të kompilohet në shumë platforma. Ekziston ende një sasi enorme software-i në C që mirëmbajtja dhe zhvillimi i tij kërkon programues që e njohin mirë këtë gjuhë.

4. **Bazë e fortë për gjuhë të tjera.**
   Nëse mësoni C-në si duhet, nuk mësoni vetëm "një sintaksë tjetër": mësoni konceptet themelore të programimit të strukturuar dhe menaxhimit të burimeve.

5. **Thjeshtësi në bërthamë.**
   Gjuha ka një bërthamë relativisht të vogël rregullash; fuqia vjen nga mënyra se si i kombinon ato.

Paralajmërim i ndershëm

C **nuk** është gjuhë që "fal" gabimet:

- një gabim me pointer-a mund të prishë të gjithë programin;
- mungon menaxhimi automatik i memorjes;
- kompiluesi shpesh ju lejon të bëni gjëra të rrezikshme.

Por pikërisht për këtë arsye, të mësuarit C ju bën programues më të kujdesshëm dhe më të vetëdijshëm.

2.6 Nga kodi burimor te programi ekzekutues

Përpara se të shohim sintaksën e C-së, është e rëndësishme të kuptojmë se **si** kthehet një file `.c` në një program që mund ta ekzekutojmë. Ky proces përbëhet nga dy faza kryesore:

Kompilimi + **Linkimi**

Në praktikë, mjeti që përdorim (p.sh. `gcc`) i kryen disa hapa njëri pas tjetrit.

1. Preprocesimi (Preprocessing)

Ky është hapi paraprak ku trajtohen direktivat që fillojnë me `#`, si:

- `#include <stdio.h>` — fut deklarimet nga një file tjetër.
- `#define PI 3.14` — zëvendëson simbolin `PI` me vlerën përkatëse.

Rezultati është një version i zgjeruar i kodit burimor (ende tekst), ku:

- janë futur header-at,
- janë zëvendësuar makrot.

2. Kompilimi (Compilation)

Në këtë hap, kompiluesi **(1)** kontrollon gabimet sintaksore dhe tipat; **(2)** përkthen kodin burimor në kod të ndërmjetëm/assembly; **(3)** gjeneron një **file objekt** (p.sh. `program.o`). Ky file objekt:

- përmban kod makine,
- por nuk është ende program i plotë i ekzekutueshëm.

3. Asamblimi (Assembling)

Në shumë mjedise, ky hap kryhet automatikisht nga kompiluesi: kodi assembly përkthehet në kod makine binar, duke formuar file-in `.o`. Për studentin fillestar mjafton të dini që:

`.c` → `.o` bëhet nga kompiluesi.

4. Linkimi (Linking)

Linkeri merr: **(1)** një ose disa file objekt (`.o`), **(2)** funksionet nga bibliotekat standarde ose të jashtme, dhe i bashkon në **një file ekzekutues** (p.sh. `program`, `program.exe`). Në këtë hap zgjidhen:

- thirrjet e funksioneve (p.sh. `printf`) me implementimet e tyre reale;
- simbolet e shpallura në file të tjerë.

Shembull: nga `program.c` te ekzekutuesi

Hapi 1: Kompilimi në file objekt

```
gcc -c program.c -o program.o
```

Hapi 2: Linkimi në program ekzekutues

```
gcc program.o -o program
```

Tani mund të ekzekutoni:

```
./program
```

Shembull me disa file

Supozoni që projekti ka dy file:

- `main.c` — funksioni `main`.
- `functions.c` — funksione ndihmëse.

Kompilimi:

```
gcc -c main.c -o main.o
gcc -c functions.c -o functions.o
```

Linkimi:

```
gcc main.o functions.o -o myprogram
```

Ekzekutimi:

```
./myprogram
```

2.7 Forma e përgjithshme e një programi në C

Tani që dimë pse ekziston C-ja dhe si kthehet një `.c` në program ekzekutues, le të shohim si duket *programi më i vogël i arsyeshëm* në C.

```
#include <stdio.h>

int main(void) {
   /* Këtu shkruajmë komandat (deklarata) */
   return 0;
}
```

Ky skelet përmbledh disa ide shumë të rëndësishme:

- `#include <stdio.h>` është një **direktivë paraprosesori**: i thotë paraprosesorit të fusë informacionin nga file-i `stdio.h` (deklarimet e `printf`, `scanf`, etj.).
- `int main(void)` është **funksioni kryesor**: aty fillon ekzekutimi i programit.
- `{` dhe `}` kufizojnë **trupin** e funksionit, pra bllokun e deklaratave që do të ekzekutohen.
- `return 0;` kthen vlerën `0` te sistemi operativ — një mënyrë formale për të thënë "programi përfundoi me sukses".

Tre shtyllat e çdo programi C

Edhe programet më të vogla në C kombinon:

1. **Direktiva** (`#include`, `#define`, ...)
2. **Funksione** (si `main`, por edhe funksione të tjerë që do të shkruajmë vetë)
3. **Deklarata** (komandat brenda funksioneve, që ekzekutohen njëra pas tjetrës)

Pjesa tjetër e kursit është, në thelb, të mësojmë t'i përdorim mirë këto tre.

Para se kompiluesi të shohë kodin tuaj, paraprosesori e "përgatit" atë. Çdo rresht që fillon me `#` lexohet nga paraprosesori.

```
#include <stdio.h>
```

Kjo do të thotë: *"Fut deklarimet nga file-i `stdio.h` këtu."* Pa këtë rresht, kompiluesi nuk e di se ekziston `printf`, prandaj do të ankohet.

2.8 Funksioni `printf`

Për të shfaqur mesazhe në ekran, ne nuk kemi komandë të brendshme `print` si në disa gjuhë të tjera; përdorim funksionin `printf` nga `stdio.h`. Forma bazë:

```
printf("format string", vlera1, vlera2, ...);
```

Argumenti i parë është një **string formati**. Ai mund të përmbajë:

- tekst të zakonshëm, që shtypet siç është;
- **specifikues formati**, që fillojnë me `%` dhe tregojnë si të shfaqen vlerat pas stringut.

Shembull i thjeshtë

```
int a = 10;
float x = 3.5f;

printf("a = %d, x = %f\n", a, x);
```

Kur ekzekutohet kodi i mëposhtëm, do të kemi këtë rezultat:

```
a = 10, x = 3.500000
```

Gabimet klasike me `printf`

`printf` *nuk* ju fal shumë gabime. Disa gjëra që duhen shmangur:

- **Mospërputhje numrash**:

```
printf("%d %d\n", a); /* GABIM: mungon një vlerë */
printf("%d\n", a, x); /* GABIM: ka një vlerë tepër */
```

- **Mospërputhje tipash**:

```
printf("%f\n", a); /* GABIM: %f pret float, jo int */
```

Kompiluesi jo gjithmonë i kap këto; shpesh programi kompilohet, por shfaq *marrëzira*. Prandaj, kujdes me stringun e formatit.

Specifikuesit e formatit

Specifikuesi i formatit është "urë" mes vlerës në memorie dhe pamjes së saj të shtypur. Më të përdorurit në këtë fazë të kursit:

Specifikuesi	Përshkrimi
`%d`	numër i plotë (`int`) në bazën 10
`%f`	numër me presje dhjetore (`float`/`double`)
`%e`	formë eksponenciale (`1.23e+04`)
`%g`	zgjedh automatikisht mes `%f` dhe `%e`
`%c`	një karakter
`%s`	një varg (string) karakteresh

Tabela 2.1: Format specifikuese

FormatSpeci

Specifikuesit mund të përfshijnë edhe **gjerësi fushe** dhe **precizion**.

```
#include <stdio.h>

int main(void) {
   int i = 40;
   float x = 839.21f;
```

```
    printf("%d|%5d|%-5d|%5.3d|\n", i, i, i, i);
    printf("%10.3f|%10.3e|%-10g|\n", x, x, x);

    return 0;
}
```

Output:

```
40| 40|40 | 040|
  839.210| 8.392e+02|839.21
```

Si t'i lexosh këto formate

- `%5d` — numër i plotë në të paktën 5 karaktere (mbushet me hapësira nëse është më i shkurtër).
- `%-5d` — e njëjta gjë, por i justifikuar majtas.
- `%5.3d` — të paktën 3 shifra (mbushet me zero nga e majta), fusha totale të paktën 5 karaktere.
- `%10.3f` — numër me presje dhjetore, 3 shifra pas pikës, fusha totale 10 karaktere.
- `%10.3e` — e njëjta logjikë, por në format eksponencial.
- `%-10g` — përdor formatin "më të shkurtër" (`f` ose `e`), fusha 10 karaktere, e justifikuar majtas.

Mos i mësoni përmendësh mekanikisht; lexoni formën si "gjerësi.precizionlloj".

2.9 Sekuencat escape

Brenda një stringu nuk mund të shkruajmë lirisht çdo karakter. Për karaktere të veçanta përdorim **sekuenca escape**: një `\` i ndjekur nga një simbol. Më të rëndësishmet për ne tani:

Sekuenca	Përshkrimi
`\n`	kalim në rresht të ri
`\t`	tab horizontale
`\\`	vetë simboli \
`\"`	thonjëza të dyfishta brenda stringut
`\'`	thonjëza të vetme
`\a`	sinjal tingulli (në disa sisteme)

Tabela 2.2: Sekuencat `escape` të zakonshme në gjuhën C

tab: escape-

Shembull me escape

```
printf("Item\tPrice\n");
printf("\"Hello!\"\n");
printf("Backslash: \\\n");
```

Output:

```
Item Price
"Hello!"
Backslash: \
```

2.10 Leximi i të dhënave: funksioni scanf

Në të njëjtën mënyrë që `printf()` përdoret për të shfaqur të dhëna në ekran, funksioni `scanf()` shërben për të **lexuar** të dhëna nga hyrja standarde (zakonisht tastiera). Si dhe `printf`, ai përdor një *format string* që përmban specifikuesit e llojit të të dhënave që priten nga përdoruesi.

Shembull — Leximi i disa vlerave

```c
int i, j;
float x, y;

scanf("%d%d%f%f", &i, &j, &x, &y);
```

Nëse përdoruesi fut:

```
1 -20 0.3 -4.0e3
```

atëherë pas ekzekutimit të `scanf`, variablat do kenë këto vlera:

```
i = 1
j = -20
x = 0.3
y = -4000.0
```

Rëndësia e simbolit &

Në thirrjen e `scanf`, çdo variabël duhet të paraprihet nga `&` (simboli "adresë e") për t'i treguar funksionit vendndodhjen e saktë të variablit në memorje. Pa këtë simbol, `scanf` nuk mund të ndryshojë vlerën e variablit në programin tuaj.

`scanf("%d", &num);`	e saktë
`scanf("%d", num);`	gabim

Nëse mungon `&`, programi mund të prodhojë rezultate të pakuptimta ose të ndërpritet papritur (crash). Ky është një nga gabimet më të zakonshme tek fillestarët.

Specifikuesi	Përshkrimi
`%d`	lexon një numër të plotë (`int`)
`%f`	lexon një numër me presje dhjetore (`float`)
`%lf`	lexon një numër `double`
`%c`	lexon një karakter të vetëm
`%s`	lexon një varg (string) karakteresh (deri në hapësirë)

Tabela 2.3: Specifikuesit më të përdorur me `scanf`

tab: scanf-specifiers

Këshilla për përdorimin e `scanf`

- **Kujdes me hapësirat:** `scanf` ndalon leximin e një `%s` kur has hapësirë, prandaj nuk mund të lexojë fjali të plota.
- **Kontrolloni vlerën e kthyer:** `scanf` kthen numrin e elementeve të lexuar me sukses. Përdoreni për të shmangur inpute të gabuara.

Shembull praktik

Programi më poshtë lexon dy numra të plotë dhe shfaq shumën e tyre:

```c
#include <stdio.h>

int main(void) {
   int a, b;
   printf("Shkruani dy numra: ");
   scanf("%d%d", &a, &b);
   printf("Shuma është: %d\n", a + b);
   return 0;
}
```

Shembull hyrjeje dhe daljeje:

```
Shkruani dy numra: 12 7
Shuma është: 19
```

2.11 Operatorët aritmetikë

Operatorët aritmetikë bazë

C ofron një grup operatorësh aritmetikë që përdoren si për konstante, ashtu edhe për variabla. Janë të njëjtët simbole që prisni nga matematika, por me disa rregulla që duhen respektuar (kryesisht kur punoni me `int`).

Operatori	Përshkrimi	Shembull
`+`	Mbledhje	`a + b`
`-`	Zbritje	`a - b`
`*`	Shumëzim	`a * b`
`/`	Pjesëtim	`a / b`
`%`	Mbetje (*modulo*)	`a % b`

Tabela 2.4: Operatorët aritmetikë në gjuhën C

tab: aritmetik

Operacione me `int`

```c
#include <stdio.h>

int main(void) {
   int a = 10, b = 3;

   printf("Mbledhja: %d\n", a + b);
   printf("Zbritja: %d\n", a - b);
   printf("Shumezimi: %d\n", a * b);
   printf("Pjestimi (int): %d\n", a / b);
   printf("Mbetja: %d\n", a % b);

   return 0;
}
```

Output-i tipik:

```
Mbledhja: 13
Zbritja: 7
Shumezimi: 30
Pjestimi (int): 3
Mbetja: 1
```

Dy vëzhgime të rëndësishme

- Kur të dy operandët janë `int`, pjesëtimi `/` heq pjesën dhjetore (`10 / 3` jep `3`, jo `3.33`).
- Operatorit `%` nuk i lejohet pjesëtimi me 0; `a % 0` ose `a / 0` çojnë në sjellje të papërcaktuar.

Operatorët e inkrementimit dhe dekrementimit

Shumë programe kanë nevojë të rrisin ose ulin një variabël me 1 (indekse, numërime, iterime). Shkrimi i plotë:

```c
i = i + 1;
j = j - 1;
```

është i saktë, por në C ekzistojnë forma më të shkurtra dhe më tipike. Kemi disa forma të shkurtra me `+=` dhe `-=`. Këto operatorë kombinojnë caktimin dhe veprimin aritmetik:

```c
i += 1; /* e njejte me i = i + 1; */
j -= 1; /* e njejte me j = j - 1; */
```

Më tej, C ofron edhe operatorët `++` dhe `-`

```c
++i; /* rrit i me 1 */
--j; /* ul j me 1 */
```

që janë semantikisht të barabartë me `i = i + 1;` dhe `j = j - 1;` kur përdoren si deklarata më vete. Operatorët `++` dhe `-` mund të vendosen:

- para variablës: `++i`, `-i` (**prefix**);
- pas variablës: `i++`, `i-` (**postfix**).

Të dyja rrisin/ulin variablën me 1; dallimi shfaqet kur rezultati përdoret brenda një shprehjeje.

Shembull — Prefix

```c
int i = 1;
printf("i = %d\n", ++i); /* rrit, pastaj perdor -> shtyp 2 */
printf("i = %d\n", i); /* shtyp 2 */
```

Shembull — Postfix

```c
int i = 1;
printf("i = %d\n", i++); /* perdor, pastaj rrit -> shtyp 1 */
printf("i = %d\n", i); /* tani i eshte 2 */
```

Shembull i kombinuar

```c
#include <stdio.h>

int main(void) {
   int i = 1, j = 2, k;

   k = ++i + j++;
   printf("i = %d, j = %d, k = %d\n", i, j, k);

   return 0;
}
```

Analizë e shkurtër:

- `++i` -> i kalon nga 1 në 2, në shprehje përdoret 2.
- `j++` -> në shprehje përdoret 2, më pas j bëhet 3.
- `k = 2 + 2 = 4`.

Në fund: `i = 2`, `j = 3`, `k = 4`.

2.12 Ushtrime

Ushtrimi 1: Formati i datës

Shkruani një program që merr nga përdoruesi një datë në formatin `mm/dd/yyyy` dhe e shfaq atë në formatin `yyyymmdd`.

```
Start
Lexo month, day, year nga input në formatin mm/dd/yyyy
Shfaq year, month, day në formatin yyyymmdd
End
```

```c
#include <stdio.h>

int main(void) {
   int month, day, year;

```

```
    printf("Enter a date (mm/dd/yyyy): ");
    scanf("%d/%d/%d", &month, &day, &year);

    printf("You entered the date %04d%02d%02d\n", year, month, day);

    return 0;
}
```

Ushtrimi 2: Formatimi i të dhënave të produktit

Programi lexon: numrin e artikullit, çmimin për njësi dhe datën e blerjes, dhe i shfaq të dhënat në kolona të rregulluara.

```
Start
Lexo item
Lexo price
Lexo month, day, year në formatin mm/dd/yyyy
Shfaq "Item" dhe item
Shfaq "Unit Price" dhe price me 2 shifra pas presjes
Shfaq "Purchase Date" dhe datën në formatin mm/dd/yyyy
End
```

```
#include <stdio.h>

int main(void) {
    int item;
    double price;
    int month, day, year;

    printf("Enter item number: ");
    scanf("%d", &item);

    printf("Enter unit price: ");
    scanf("%lf", &price);

    printf("Enter purchase date (mm/dd/yyyy): ");
    scanf("%d/%d/%d", &month, &day, &year);

    printf("\nItem %d\n", item);
    printf("Unit Price $ %6.2f\n", price);
    printf("Purchase Date %02d/%02d/%04d\n", month, day, year);

    return 0;
}
```

Ushtrimi 3: Ndarja e ISBN-13

Lexoni një ISBN-13 në formatin `xxx-x-xxx-xxxxx-x` dhe shfaqni secilën pjesë veçmas.

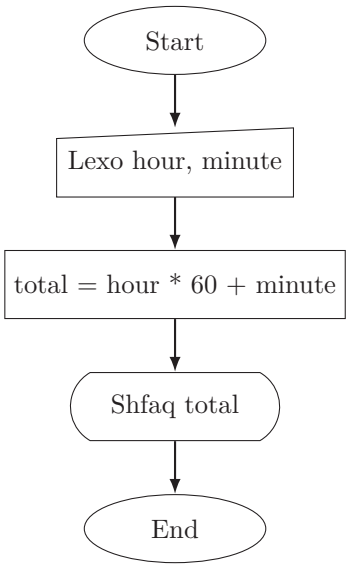
```
Start
Lexo gs1, group, publisher, item, check nga input në formatin xxx-x-xxx-xxxxx-x
Shfaq "GS1 prefix" dhe gs1
Shfaq "Group identifier" dhe group
Shfaq "Publisher code" dhe publisher
Shfaq "Item number" dhe item
Shfaq "Check digit" dhe check
End
```

```c
#include <stdio.h>

int main(void) {
   int gs1, group, publisher, item, check;

   printf("Enter ISBN: ");
   scanf("%d-%d-%d-%d-%d", &gs1, &group, &publisher, &item, &check);

   printf("GS1 prefix: %d\n", gs1);
   printf("Group identifier: %d\n", group);
   printf("Publisher code: %d\n", publisher);
   printf("Item number: %d\n", item);
   printf("Check digit: %d\n", check);

   return 0;
}
```

Ushtrimi 4: Minutat nga mesnata

Shkruani një program që lexon një kohë në formatin `hh:mm` (dy numra të plotë, orë dhe minuta) dhe shfaq numrin e minutave që kanë kaluar nga mesnata.

```
Start
Lexo hour, minute në formatin hh:mm
total = hour * 60 + minute
Shfaq "Minutes since midnight:" dhe total
End
```




```c
#include <stdio.h>

int main(void) {
   int hour, minute;
   int total;

   printf("Enter a 24-hour time (hh:mm): ");
   scanf("%d:%d", &hour, &minute);
   total = hour * 60 + minute;

   printf("Minutes since midnight: %d\n", total);

   return 0;
}
```

Ushtrimi 5: Përdorimi i ++ dhe -

Shkruani një program që lexon një numër të plotë `n` dhe demonstruen ndryshimin midis përdorimit të `n++` dhe `++n`.

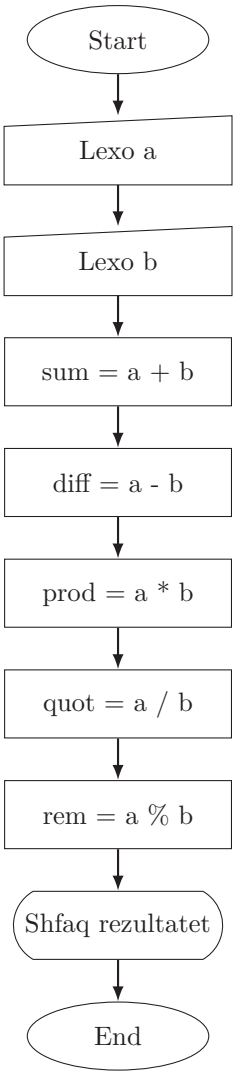
```
Start
Lexo n
Shfaq n
Shfaq vlerën e n++ (përdor n, pastaj rrit)
Shfaq vlerën aktuale të n
Shfaq vlerën e ++n (rrit, pastaj përdor)
Shfaq vlerën finale të n
End
```

```c
#include <stdio.h>

int main(void) {
   int n;

   printf("Enter an integer: ");
   scanf("%d", &n);

   printf("Initial n: %d\n", n);
   printf("Using n++ (prints old n): %d\n", n++);
   printf("After n++: %d\n", n);
   printf("Using ++n (increments then prints): %d\n", ++n);
   printf("Final n: %d\n", n);

   return 0;
}
```

Ushtrimi 6: Operacione aritmetikore me dy numra

Shkruani një program që lexon dy numra të plotë `a` dhe `b` dhe shfaq: shumën, diferencën, prodhimin, pjesëtimin e plotë dhe mbetjen `a % b`.

```
Start
Lexo a
Lexo b
sum = a + b
diff = a - b
prod = a * b
quot = a / b (pjesëtim int, supozojmë b != 0)
rem = a % b
Shfaq sum, diff, prod, quot, rem
End
```




```c
#include <stdio.h>

int main(void) {
   int a, b;
   int sum, diff, prod, quot, rem;

   printf("Enter two integers (a b): ");
   scanf("%d%d", &a, &b);

   sum = a + b;
   diff = a - b;
   prod = a * b;
   quot = a / b; /* supozojmë b != 0 për këtë ushtrim */
   rem = a % b;

   printf("Sum: %d\n", sum);
   printf("Difference: %d\n", diff);
   printf("Product: %d\n", prod);
   printf("Quotient: %d\n", quot);
   printf("Remainder: %d\n", rem);

   return 0;
}
```

2.13 Detyrat e shtëpisë

Detyra 1: Formati i kohës

Shkruani një program që lexon një kohë në formatin `hh:mm` (ora 24-orëshe) dhe e shfaq atë në formatin:

- `hhmm` (pa pikëpresje),
- dhe një mesazh shoqërues, p.sh. `"Ora e futur është:  14:05"`.

Kërkohet:

a) përshkruani algoritmin me pseudokod;

b) implementoni programin në C duke përdorur `scanf` dhe `printf` me formatim të përshtatshëm.

Detyra 2: Fatura e thjeshtë

Një dyqan shet një produkt me çmim për njësi. Përdoruesi fut: **(1)** emrin e produktit (pa hapësira), **(2)** çmimin për njësi (numër me presje dhjetore), **(3)** sasinë (numër i plotë). Programi duhet të shfaqë një "faturë" të thjeshtë në tre rreshta:

- emri i produktit,
- sasia,
- totali (`sasia * çmimi`) i shfaqur me dy shifra pas presjes.

Kërkohet:

a) pseudokodi i algoritmit;

b) bllokskema që tregon qartë hapat e leximit dhe llogaritjes;

c) programi në C, duke përdorur formatet e duhura të `scanf` dhe `printf`.

Detyra 3: Konvertimi i temperaturës

Shkruani një program që lexon një temperaturë në gradë Celsius (`C`) dhe llogarit temperaturën përkatëse në Fahrenheit (`F`) duke përdorur formulën:

$$F = \frac{9}{5} \cdot C + 32$$

Programi duhet të shfaqë të dy vlerat me një mesazh të qartë, p.sh.: `"C = 20.0, F = 68.0"`. **Kërkohet:**

a) përshkruani algoritmin në pseudokod;

b) vizatoni bllokskemën;

c) shkruani programin në C duke përdorur `float` ose `double` dhe formatim korrekt në `printf`.

Detyra 4: Shuma dhe mesatarja e tre numrave

Programi duhet të lexojë tre numra të plotë dhe të shfaqë: **(1)** shumën e tyre; **(2)** mesataren aritmetike (si numër me presje dhjetore). **Kërkohet:**

a) shkruani pseudokodin;

b) vizatoni bllokskemën;

c) shkruani kodin në C duke pasur kujdes që mesatarja të mos llogaritet me pjesëtim vetëm `int/int`.

2.14 Përmbledhje

Në këtë kapitull kaluam nga "mendimi algoritmik" te *zbatimi konkret* në një gjuhë reale programimi: C-ja. Qëllimi nuk ishte të mësoni të gjitha rregullat e C-së, por të ndërtoni bazën e duhur për pjesën tjetër të kursit.

1. **Struktura minimale e një programi në C.**
   - `#include <stdio.h>` për të përdorur `printf`, `scanf`, etj.
   - `int main(void){...}` si hyrja kryesore e programit.
   - Kllapat `{ }` përcaktojnë blloqe; brenda tyre kemi deklarata që ekzekutohen me radhë.
   - `return 0;` sinjalizon përfundim të suksesshëm te sistemi operativ.
2. **Dalja në ekran: `printf`.**
   - Përdor *format string* me specifikues si `%d`, `%f`, `%s`, `%c`, ...
   - Mund të përcaktojmë gjerësi fushe dhe precizion (`%5d`, `%10.3f`, ...).
   - Gabimet tipike: numër i gabuar argumentesh, tip i gabuar kundrejt formatit ⇒ output i pakuptimtë, jo domosdoshmërisht gabim kompajlimi.
3. **Sekuencat `escape`.**
   - `\n`, `\t`, `\¨`, `\\`, ...
   - Përdoren për kontroll formati brenda stringjeve dhe për karaktere speciale.
4. **Leximi i të dhënave: `scanf`.**
   - Lexon sipas një format string: `"%d%f%s"`....
   - Kërkon adresën e variablave: `&x`, `&i`, ... (përveç disa rasteve si `char[]`).
   - Harrimi i `&` ose mospërputhja e formatit me tipin çojnë në sjellje të rrezikshme.
   - `%s` ndalon te hapësira: jo i përshtatshëm për fjali të plota pa teknika shtesë.
5. **Operatorët aritmetikë dhe `++`/`-`.**
   - Operatorët bazë: `+`, `-`, `*`, `/`, `%`.
   - Me dy `int`, `/` bën pjesëtim të plotë (`7/3` ⇒ `2`); `/0` dhe `%0` janë të ndaluara.
   - Forma të shkurtra: `+=`, `-=`, etj.
   - `++i` / `i++`, `-i` / `i-` rrisin/ulin me 1; dallimi ka rëndësi brenda shprehjeve.
   - Shprehje të ndërlikuara me shumë `++`/`-` në të njëjtën rresht mund të kenë sjellje të papërcaktuar: duhen shmangur.

Appendices

SHTESA A

Detyrat e shtëpisë - Zgjidhjet

Këtu janë zgjidhjet për detyrat e kapitullit **"Hyrje në algoritme dhe programim"**. (Zgjidhjet e kapitujve të tjerë shtohen në seksione të reja.)

A.1 Hyrje në algoritme dhe programim

Detyra 1: A është algoritëm i mirë?

a) **"Hap librin ku të duash dhe lexo derisa të lodhesh."**
Jo algoritëm i vlefshëm. Nuk ka kriter të qartë përfundimi (*"derisa të lodhesh"* është subjektive), nuk ka output të përcaktuar, udhëzimet janë të paqarta.

b) **"Merr tre numra, gjej shumën e tyre dhe pjesëto me 3. Shfaq rezultatin."**
Algoritëm i vlefshëm. Ka input të qartë (tre numra), hapa të qartë dhe fundorë, si dhe output të përcaktuar (mesatarja).

c) **"Përsërit: shto 1 numrit fillestar."**
Jo algoritëm i vlefshëm. Nuk ka kusht ndalimi, pra nuk është fundor; nuk ka as output të përcaktuar.

d) **"Hidh zarin derisa të dalë 6; numëro sa herë u hodh zar."**
Algoritëm i vlefshëm. Ka input (hedhje zari), kusht të qartë ndalimi (del 6), hapa të qartë dhe output (numri i hedhjeve).

Detyra 2: Rregullo algoritmin e paqartë

a) Pse nuk është algoritëm i mirë?

- Përdor terma të paqartë: "pak ujë", "pak lëng", "të ftohtë".
- Nuk jep sasi, kohë, as hapa të saktë.
- Nuk dihet qartë kur përfundon (sa minuta në frigorifer? sa përzierje?).

b) Version i përmirësuar (me fjali të qarta)

1. Merr një gotë me kapacitet 250 ml.
2. Hidh 150 ml ujë në gotë.
3. Shto 100 ml lëng portokalli.
4. Përziej për 10 sekonda.
5. Vendos gotën në frigorifer për 15 minuta.
6. Nxirre gotën nga frigoriferi dhe shërbe pijen.

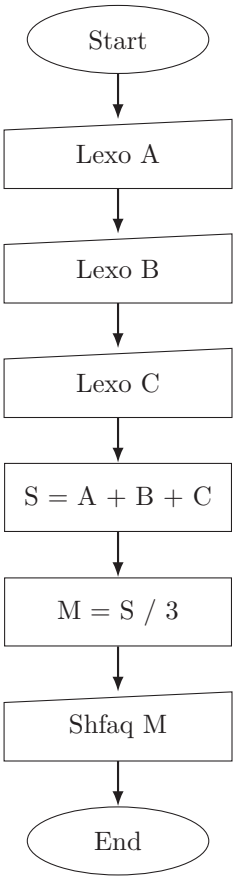
c) Pseudokodi

```
Start
Merr gotë 250 ml
U = 150 // ujë në ml
L = 100 // lëng në ml
Hidh U në gotë
Hidh L në gotë
Përziej për 10 sekonda
Vendos gotën në frigorifer për 15 minuta
Nxirr gotën
Shfaq "Pija është gati"
End
```

Detyra 3: Mesatarja e tre numrave

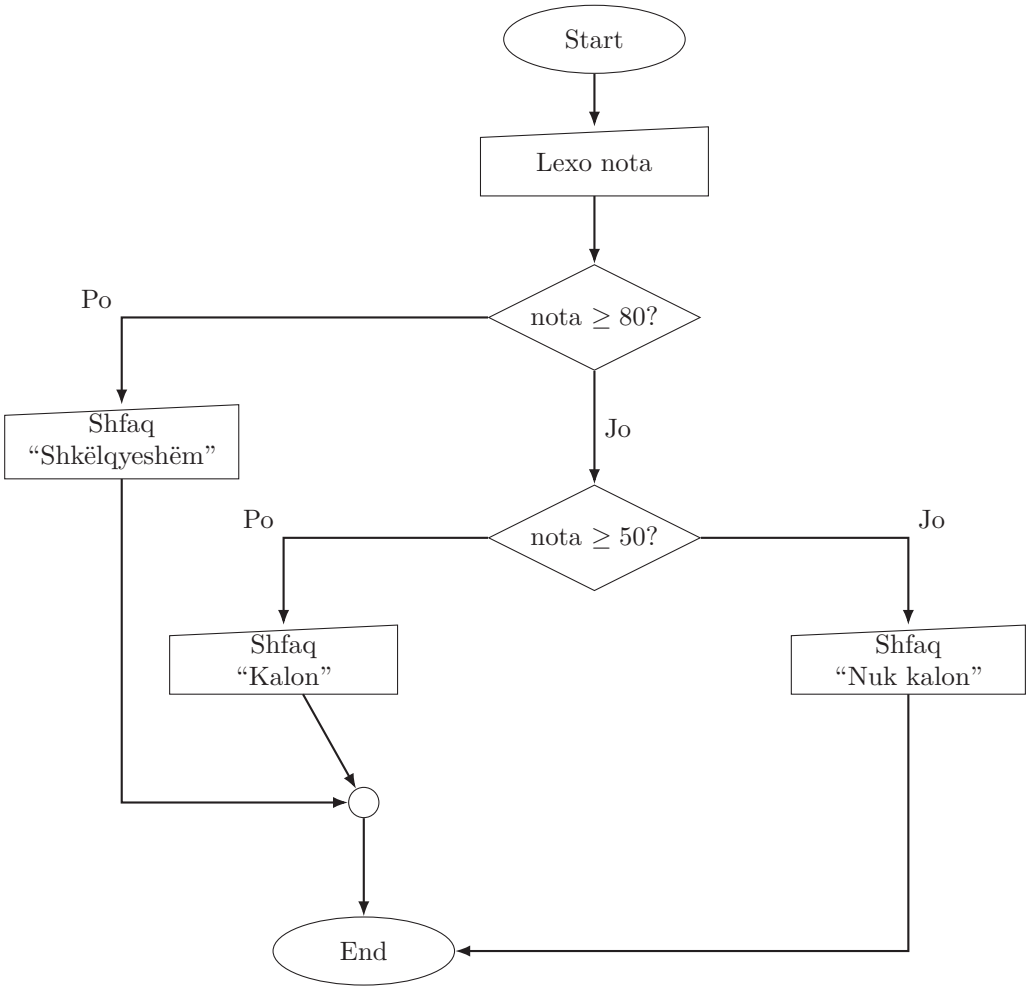
1. Lexo tre numra A, B, C.
2. Llogarit shumën $S = A + B + C$.
3. Llogarit mesataren $M = S/3$.
4. Shfaq mesataren M.

```
Start
Lexo A
Lexo B
Lexo C
S = A + B + C
M = S / 3
Shfaq "Mesatarja është:", M
End
```



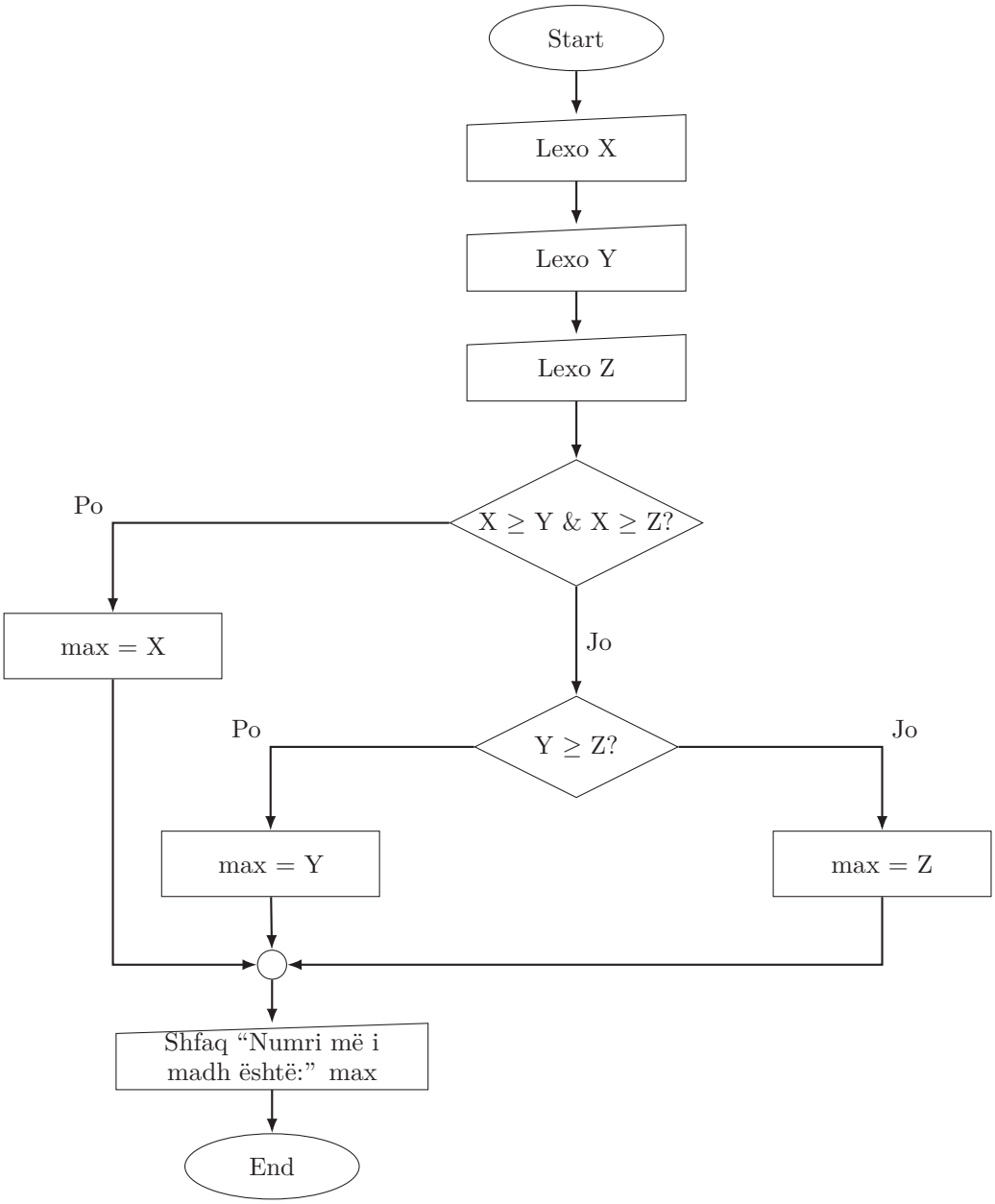
Detyra 4: Vendos nëse studenti kalon (i zgjeruar)

```
Start
Lexo nota
Nëse nota >= 80 atëherë
   Shfaq "Shkëlqyeshëm"
Përndryshe nëse nota >= 50 atëherë
   Shfaq "Kalon"
Përndryshe
   Shfaq "Nuk kalon"
FundNese
End
```



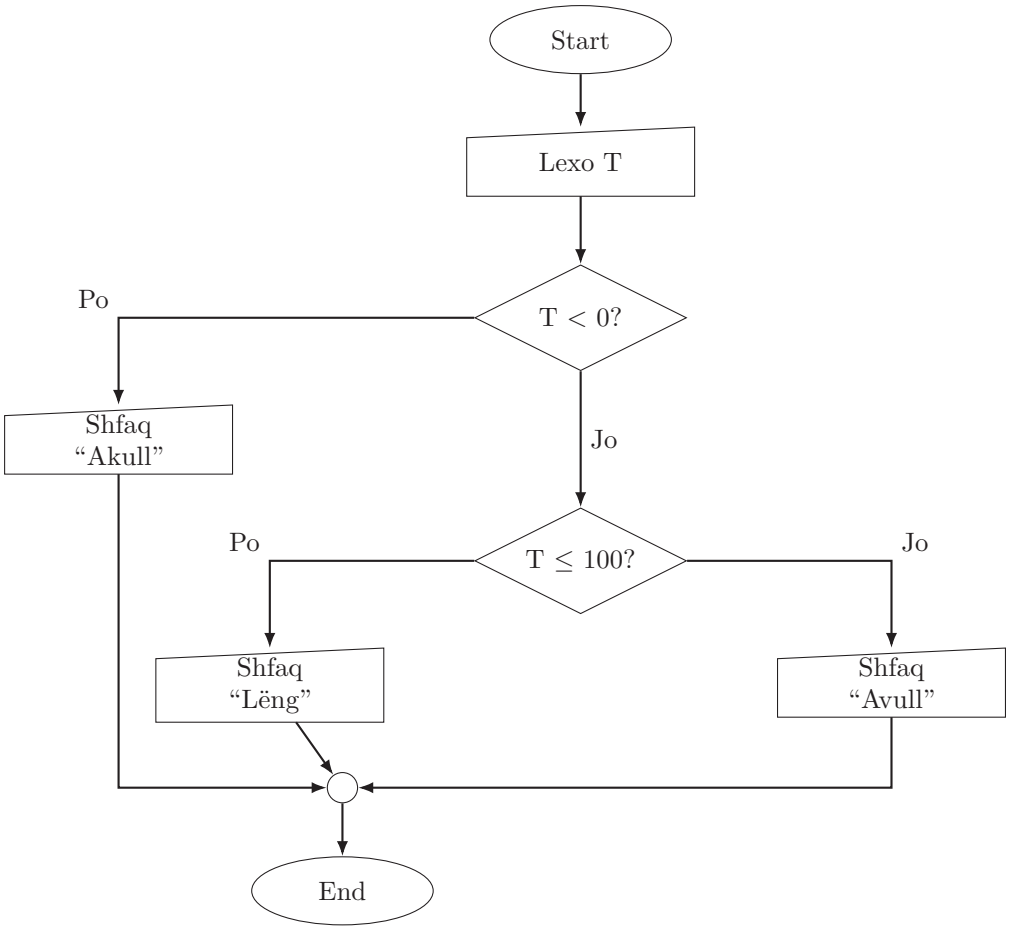
Detyra 5: Numri më i madh nga tre numra

```
Start
Lexo X
Lexo Y
Lexo Z
Nëse X >= Y dhe X >= Z atëherë
   max = X
Përndryshe nëse Y >= Z atëherë
   max = Y
Përndryshe
   max = Z
FundNese
Shfaq "Numri më i madh është:", max
End
```



Detyra 6: Temperatura e ujit

```
Start
Lexo T
Nëse T < 0 atëherë
   Shfaq "Akull"
Përndryshe nëse T <= 100 atëherë
   Shfaq "Lëng"
Përndryshe
   Shfaq "Avull"
FundNese
End
```



A.2 Hyrje në gjuhën C

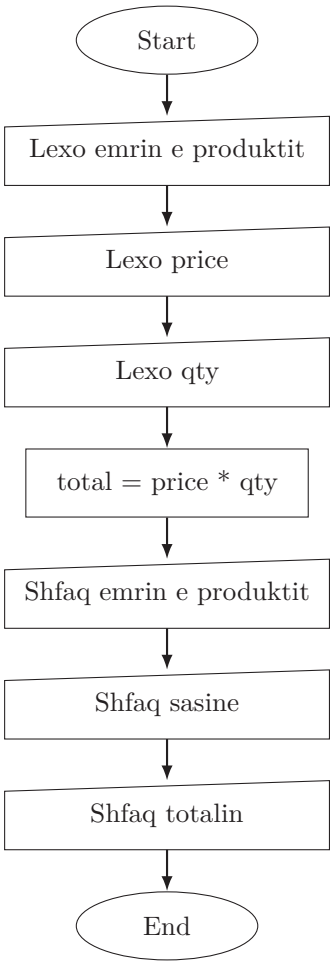
Detyra 1: Formati i kohës

```
Start
Lexo hour, minute nga inputi ne formatin hh:mm
Shfaq "Ora e futur është: ", hour, ":", minute (me 2 shifra secila)
Shfaq "Ne format hhmm: ", hour, minute (pa pikespresje, me 2 shifra secila)
End
```

```c
#include <stdio.h>

int main(void) {
   int hour, minute;

   printf("Enter time (hh:mm): ");
   scanf("%d:%d", &hour, &minute);

   printf("Ora e futur eshte: %02d:%02d\n", hour, minute);
   printf("Ne format hhmm: %02d%02d\n", hour, minute);

   return 0;
}
```

Detyra 2: Fatura e thjeshtë

```
Start
Lexo emrin e produktit (pa hapesira)
Lexo cmimin per njesi (price)
Lexo sasine (qty)
total = price * qty
Shfaq emrin e produktit
Shfaq sasine
Shfaq totalin me 2 shifra pas presjes
End
```

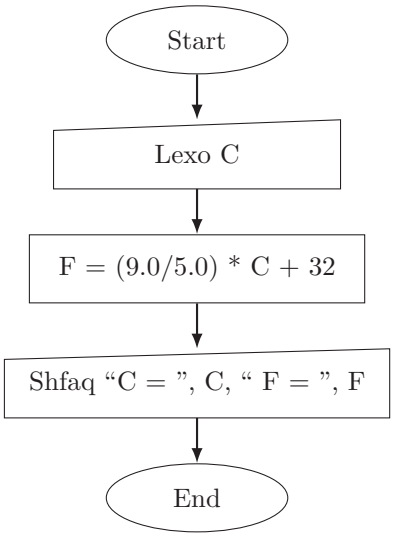



```c
#include <stdio.h>

int main(void) {
   char product[31];
   double price, total;
   int quantity;

   printf("Enter product name (no spaces): ");
   scanf("%30s", product);

   printf("Enter unit price: ");
   scanf("%lf", &price);

   printf("Enter quantity: ");
   scanf("%d", &quantity);

   total = price * quantity;

```

```
    printf("\nProduct : %s\n", product);
    printf("Quantity: %d\n", quantity);
    printf("Total : %.2f\n", total);

    return 0;
}
```

Detyra 3: Konvertimi i temperaturës

```
Start
Lexo C
F = (9 / 5) * C + 32 // perdor 9.0/5.0 per llogaritje me presje dhjetore
Shfaq "C = ", C, ", F = ", F
End
```

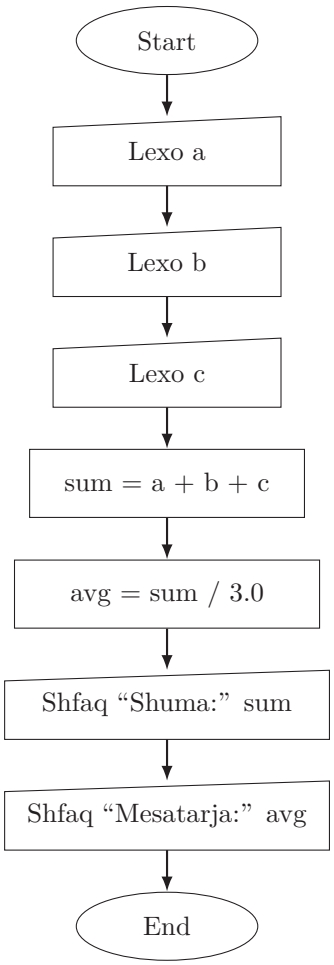



```
#include <stdio.h>

int main(void) {
    double C, F;

    printf("Enter temperature in Celsius: ");
    scanf("%lf", &C);

    F = (9.0 / 5.0) * C + 32.0;

    printf("C = %.1f, F = %.1f\n", C, F);

    return 0;
}
```

Detyra 4: Shuma dhe mesatarja e tre numrave

```
Start
Lexo a
Lexo b
Lexo c
sum = a + b + c
avg = sum / 3.0 // sigurohu qe te jete pjesetim me presje dhjetore
Shfaq "Shuma:", sum
Shfaq "Mesatarja:", avg
```

```
End
```




```c
#include <stdio.h>

int main(void) {
   int a, b, c;
   int sum;
   double avg;

   printf("Enter three integers: ");
   scanf("%d%d%d", &a, &b, &c);

   sum = a + b + c;
   avg = sum / 3.0;

   printf("Shuma: %d\n", sum);
   printf("Mesatarja: %.2f\n", avg);

   return 0;
}
```

SHTESA B

Laboratorët